

Cala's Pokecom BASIC System

Version 0.94

Programming Reference Manual /
プログラミング・リファレンスマニュアル

for ClockworkPi PicoCalc
with Raspberry Pi Pico 2 W

Copyright (C) 2026 Cala Maclir

目次

1. BASIC Language Overview	11
2. Lexical Elements	11
3. Data Types, Variables, Arrays	11
DIM	12
4. Operator Precedence	12
整数除算・ビット演算 (v0.93)	12
5. Assignment, PRINT, INPUT	13
LET / Assignment	13
PRINT	13
INPUT	13
6. Conditional and Loops	13
IF ... THEN ... ELSE	13
FOR ... NEXT	13
WHILE ... WEND	14
DO ... LOOP [UNTIL]	14
7. Branch and Termination (番号分岐はClassic専用)	14
8. Timing and Input	14
SLEEP	14
PAUSE	14
INKEY	14
TIMER	14
INKEY Key Code	15

9. Numeric Functions	16
10. String Functions	17
11. Graphics Reference	17
12. PCG Reference	18
GDEF	18
GPALETTE	18
13. Audio Reference	18
BEEP	18
PLAY	18
MML	19
WAV / MP3	19
14. I2C Reference	19
I2C SCAN	19
I2CREAD	19
I2CWRITE	20
15. Other BASIC Statements	20
16. REPL Command Reference	20
v0.92 storage / compile behavior	21
17. Runtime Limits and Errors	21
18. Quick Reference	22
Statements (alphabetical)	22
Functions (alphabetical)	22
19. Structured BASIC	22
EXIT FOR / EXIT DO (Structured専用)	23

SELECT CASE / CASE / CASE ELSE / END SELECT (Structured専用)	23
FUNCTION / END FUNCTION / RETURN	24
20. 実装上限と非対応構文	25
21. DATA / READ / RESTORE (Classic / Structured)	25
22. Statement / Function / REPL 個別リファレンス	26
Assignment / LET	26
PRINT	26
INPUT	26
DIM	26
REM / apostrophe comment	27
Single-line IF	27
Block IF / ELSEIF / ELSE / END IF	27
FOR / NEXT	27
WHILE / WEND	27
DO / LOOP	27
EXIT FOR	28
EXIT DO	28
GOTO	28
GOSUB	28
RETURN - Classic GOSUB return	28
ON ... GOTO	29
ON ... GOSUB	29
SELECT CASE	29

CASE	29
CASE ELSE	30
END SELECT	30
FUNCTION	30
END FUNCTION	30
GLOBAL	30
RETURN expression - Structured FUNCTION return	31
DATA	31
READ	31
RESTORE	31
END	31
STOP	31
SLEEP	32
PAUSE	32
RANDOMIZE	32
LOCATE	32
SCREEN	32
CLS statement	32
COLOR	33
COLORHSV	33
PSET	33
LINE	33
CIRCLE	33
BOX	33

PAINT	33
FLUSH	34
GLOCATE	34
GPRINT	34
GDEF	34
GPALETTE	34
LOADIMAGE	34
SAVEIMAGE / SAVE IMAGE	35
BEEP	36
PLAY	36
WAVPLAY	36
WAVPAUSE	36
WAVRESUME	37
WAVSTOP	37
I2C SCAN	37
I2CWRITE	37
Operators \, <<, >>, XOR	37
ABS	37
INT	37
VAL	38
STR\$	38
LEN	38
CHR\$	38
ASC	38

LEFT\$	38
RIGHT\$	38
MID\$	39
INSTR	39
STRING\$	39
SPC	39
TAB	39
RND	39
RNDI	39
SIN	39
COS	40
TAN	40
SQR	40
ATN	40
LOG	40
LN	40
ASIN	40
ACOS	41
ATAN2	41
EXP	41
PI	41
RAD	41
DEG	41
SGN	41

MIN	.41
MAX	.42
CLAMP	.42
TIMER	.42
INKEY	.42
I2CREAD	.42
PLAYING	.42
POINT	.42
EDIT	.43
LIST	.43
RUN	.43
NEW	.43
CLEAR	.43
CLS command	.43
LOAD	.44
SAVE command	.44
FILES	.44
DIR	.44
SD	.44
SCREENSHOT	.44
XRCV	.44
XSEND	.44
YRCV	.45
YSEND	.45

DATE	45
TIME	45
DATETIME	45
SERIAL	45
CONSOLE	45
PROFILE	46
STANDBY	46
MENU	46
DIAGNOSTICS	46
HELP	46
22.1. v0.94 日時・文字列関数	46
DATES	46
TIMES	47
TRIMS	47
LTRIMS	47
RTRIMS	47
REPLACES	47
SPACES	48
22.2. v0.94 共通REPL commands	48
INFO	48
LASTERROR	48
23. Quick Reference (alphabetical)	49
Statements	49
Functions	49

Operators	49
REPL commands	49

for ClockworkPi PicoCalc with Raspberry Pi Pico 2 W

Copyright (C) 2026 Cala Maclir

本書はv0.94正式版の言語仕様です。Classic／Structuredの対応バッジ、例、期待結果を各項目に示します。LOAD／SAVEはProgram用REPL command、LOADIMAGE／SAVEIMAGEは画像用BASIC statement、SAVE IMAGEは互換構文です。最終実機試験の状況はRelease checklistを参照してください。

1. BASIC Language Overview

CPB BASICは行番号付きClassic program、行番号なしStructured program、即時実行するDirect modeを持ちます。Structured programはEditorで作成・編集します。

Classic BASICはv0.93でも継続して利用できます。Structured BASICはClassicを置き換えるものではなく、用途に応じて選べる追加モードです。Structuredの編集ではTabでindent、Alt+U（Bluetooth keyboardはShift+Tabも可）でOutdent、Alt+Mで対応blockへ移動、Alt+RでEditorから直接RUNできます。詳細はSystem Manualを参照してください。

```
10 PRINT "HELLO"  
20 END
```

- keywordとidentifierはcase-insensitiveです。
- statementはcolon (:) で区切れます。
- commentは **REM** またはapostrophe (') から行末までです。
- Classicのline numberは0～2,147,483,647です。
- 同じline numberの入力は既存lineを置換します。line numberだけなら削除します。
- programの保存上限はProgram Storage backendに従います。PSRAM利用時のINTERNAL RAMとSDはともに1,024行・本文2,047 charactersです。PSRAMなしのINTERNAL SRAM fallbackは256行・本文191 charactersです。
- **EDIT**でFull-Screen BASIC Editorを開けます。PSRAM INTERNAL／SDの2,047文字lineはEditorまたはPCで編集できます。Direct modeと通常prompt入力は191文字です。

2. Lexical Elements

要素	構文／範囲
decimal number	12、-3.5、.25
hexadecimal number	&HFF、&h20。0～0x7fffffff
string literal	"TEXT"。quoteのescape syntaxはありません
identifier	英字または_で開始し、英数字・_・\$を続ける
string variable	末尾\$。例：NAME\$
separator	colon :
expression separator	comma ,
PRINT separator	comma , またはsemicolon ;

3. Data Types、Variables、Arrays

数値はsingle-precision floating pointとして扱います。文字列variableはidentifier末尾に\$を付けます。数値と文字列を混在して演算することはできません。ただし + は文字列連結にも使用します。

```

10 A=12.5
20 NAME$="CPB"
30 PRINT NAME$+" ";A

```

DIM

分類 Array Statement 構文 **DIM** name(size[,size])[,name(size[,size])]... 説明 1Dまたは2D arrayを確保します。string arrayはname末尾に\$を付けます。例

```
10 DIM A(99),M$(9,9)
```

制限

種類	合計上限
numeric array cells	4,096
string array cells	512
string array element	127 characters

添字とsizeは数値expressionです。Direct modeのarrayはcommand完了後にresetされます。

4. Operator Precedence

同じ優先順位のbinary operatorは左から評価されます。指数演算 ^ もVersion 0.93でも左結合です。

優先順位（高→低）	operator
1	primary、function call、parentheses
2	unary +、-、NOT
3	^
4	*, /、\、MOD
5	+, -
6	<<, >>
7	=, <>, <, <=, >, >=
8	AND
9	XOR
10	OR

comparisonの結果はTRUE=-1、FALSE=0です。**AND** / **OR** / **NOT** は従来どおりlogical operator（0以外を真とする）です。**XOR** だけは32-bit bitwise operatorです。operandは数値でなければなりません。

整数除算・ビット演算（v0.93）

operator	例	結果・意味
\	17.9\5.2 / -17\5	3 / -3。各operandを0方向に整数化してから整数除算
<<	1<<4	16。32ビットでwrapする左シフト
>>	-9>>1	-5。符号ビットを保持する算術右シフト
XOR	5 XOR 3	6。32-bit bitwise XOR

4種類ともoperandをfiniteか確認し、小数部を0方向に切り捨て、signed int32範囲を確認してから演算します。NaN、Infinity、範囲外は**INTEGER RANGE ERROR**です。floatでは**2147483647**も**2147483648**に丸められるため

範囲外です。最大の入力可能な正整数floatは2147483520です。

整数化後の除数が0ならDIVISION BY ZERO、-2147483648\1はINTEGER OVERFLOWです。shift countも同じ変換を適用し、整数化後が0~31でなければSHIFT COUNT ERRORです。左シフト結果とXOR結果はtwo's-complementのsigned int32として解釈します。右シフトのcount=0は元の値です。

保存精度の制限：数値stack・variable・arrayは引き続きBasicNumber=floatです。整数演算結果もfloatへ戻ります。整数をすべて厳密に保持できる範囲は $|value| \leq 2^{24}$ (16,777,216) で、それを超えると低位ビットが失われる場合があります。例えば16777216 XOR 1はfloat保存時に16777216に戻ります。結果が 2^{31} へ丸められた場合、次の整数演算ではINTEGER RANGE ERRORになります。真のINTEGER型やA%はありません。

Classic / Structured / Direct modeで同じ意味です。2+10\3*4は14、1+2<<3は24、1<<2+1は8です。

5. Assignment、PRINT、INPUT

LET / Assignment

分類 Assignment Statement 構文 [LET] variable=expression 説明 scalar variableまたはarray elementへ代入します。LETは省略できます。

```
10 LET A=10
20 A$="PICO"
30 A(2)=A*2
```

PRINT

分類 Output Statement 構文 PRINT [expression[;], expression]... 説明 expressionをconsoleへ出力します。argumentなしは改行します。semicolonで後続出力を連結し、commaはtabulated fieldへ進めます。行末semicolonは改行しません。

```
10 PRINT "X=";12
20 PRINT "A", "B", "C"
30 PRINT SPC(3);"OK"
```

INPUT

分類 Input Statement 構文 INPUT ["prompt";] variable 説明 consoleから1個のscalar variableを読み込みます。variableの型に従って数値または文字列へ変換します。例 INPUT "NAME";N\$

6. Conditional and Loops

IF ... THEN ... ELSE

分類 Conditional Statement 構文 IF expression THEN statement-or-line [ELSE statement-or-line] 説明 数値 expressionが真のときTHEN側、偽のときELSE側を実行します。THEN／ELSEの後にはline numberまたは1個のinline statementを置けます。

```
10 IF A>0 THEN PRINT "PLUS" ELSE PRINT "ZERO OR MINUS"
20 IF A=1 THEN 100
```

FOR ... NEXT

分類 Loop Statement 構文 FOR variable=start TO limit [STEP increment] / NEXT [variable] 説明 numeric loopです。STEP省略時は1です。制限 compile時のnesting上限は16です。

```
10 FOR I=1 TO 10 STEP 2
20 PRINT I
30 NEXT I
```

WHILE ... WEND

分類 Loop Statement 構文 **WHILE expression** / **WEND** 説明 expressionが真の間、bodyを繰り返します。制限 compile時のnesting上限は16です。

DO ... LOOP [UNTIL]

分類 Loop Statement 構文 **DO** / **LOOP [UNTIL expression]** 説明 UNTILなしでは無限loop、UNTILありでは expressionが真になるまで繰り返します。制限 compile時のnesting上限は16です。

```
10 DO
20 K=INKEY
30 LOOP UNTIL K=27
```

7. Branch and Termination（番号分岐はClassic専用）

名称	分類	構文	説明
GOTO	Branch	GOTO line	指定lineへ移動
GOSUB	Subroutine	GOSUB line	subroutineを呼び出す
RETURN	Subroutine	RETURN	GOSUBの呼出元へ戻る
ON GOTO	Branch	ON expression GOTO line[,line...]	1始まりのselectorで移動先を選択
ON GOSUB	Subroutine	ON expression GOSUB line[,line...]	1始まりのselectorでsubroutineを選択
END	Termination	END	programを通常終了
STOP	Termination	STOP	programを停止

BREAKは実行中programを停止し、background audioも停止します。END、STOP、通常のruntime errorはbackground audioを自動停止しません。

8. Timing and Input

SLEEP

分類 Timing Statement 構文 **SLEEP milliseconds** 説明 指定時間待機します。background serviceを継続し、BREAKを受け付けます。例 **SLEEP 100**

PAUSE

分類 Input Statement 構文 **PAUSE** 説明 画面を変更せず通常keyを待ちます。

INKEY

分類 Numeric Function 構文 **INKEY** または **INKEY()** 戻り値 入力なしは0、入力ありはkey code。説明 non-blockingでkeyを読みます。

TIMER

分類 Numeric Function 構文 **TIMER** または **TIMER()** 戻り値 system startからの経過時間。

INKEY Key Code

key	code
Enter	0x0A
Carriage Return	0x0D
Escape	0xB1
Left	0xB4
Up	0xB5
Down	0xB6
Right	0xB7
Home	0xD2
Delete	0xD4
printable ASCII	ASCII code

physical keyboardとBluetooth HID keyboardはCPBの共通input code体系へ変換されます。

9. Numeric Functions

名称	構文	戻り値・説明
ABS	ABS(x)	absolute value
INT	INT(x)	integer directionへの変換
VAL	VAL(string\$)	stringを数値へ変換
RND	RND / RND()	pseudo-random number
RNDI	RNDI(limit)	integer random value
SIN	SIN(x)	sine
COS	COS(x)	cosine
TAN	TAN(x)	tangent
SQR	SQR(x)	square root
ATN	ATN(x)	arctangent
LOG	LOG(x)	常用対数（底10）。 $x > 0$
LN	LN(x)	自然対数（底e）。 $x > 0$
ASIN	ASIN(x)	arcsine。 $-1 \leq x \leq 1$ 、戻り値radian
ACOS	ACOS(x)	arccosine。 $-1 \leq x \leq 1$ 、戻り値radian
ATAN2	ATAN2(y,x)	2引数arctangent。引数順Y,X、戻り値radian
EXP	EXP(x)	exponential
PI	PI / PI()	π
RAD	RAD(degrees)	degreeをradianへ変換
DEG	DEG(radians)	radianをdegreeへ変換
SGN	SGN(x)	sign (-1、0、1)
MIN	MIN(a,b)	小さい方
MAX	MAX(a,b)	大きい方
CLAMP	CLAMP(value,low,high)	rangeへclamp
TIMER	TIMER / TIMER()	elapsed timer
INKEY	INKEY / INKEY()	non-blocking key input
I2CREAD	I2CREAD(address,register)	I2C read value
PLAYING	PLAYING()	PLAYがactiveなら真
POINT	POINT(x,y)	graphics pixelがnon-blackなら真

v0.93 Compatibility Change: LOG(x) changed from natural logarithm to base-10 logarithm. Use LN(x) for the previous LOG(x) behavior.

v0.92以前のLOGは自然対数です。v0.93以降はLOGが常用対数、LNが自然対数です。旧programで自然対数を計算していたY=LOG(X)はY=LN(X)へ変更してください。公開済みv0.92 PDFはその版の仕様のままです。

LOG / LNの $x \leq 0$ 、ASIN / ACOSの範囲外、これらの関数やATAN2への非finite入力はDOMAIN ERRORです。ATN(x)は従来どおり1引数arctangentです。ATAN2(0,0)は0となります（signed zeroは標準atan2に従います）。

functionを式として使用する場合、zero-argument function以外はparenthesesが必要です。

```
10 A=MAX(ABS(-5),INT(2.9))
20 PRINT SIN(RAD(90))
```

10. String Functions

名称	構文	戻り値・説明
STR\$	STR\$(number)	数値の文字列表現
LEN	LEN(string\$)	character数
CHRS	CHRS(code)	codeから1文字を作る
ASC	ASC(string\$)	先頭文字のcode
LEFT\$	LEFT\$(string\$,length)	左からlength文字
RIGHT\$	RIGHT\$(string\$,length)	右からlength文字
MID\$	MID\$(string\$,start[,length])	startから部分文字列
INSTR	INSTR(string\$,find\$)	部分文字列の位置
STRING\$	STRING\$(count,string\$)	stringをcount回反復
SPC	SPC(count)	count個のspace
TAB	TAB(column)	PRINT用tabulation string

```
10 A$=MID$( "PICO CALC", 5, 4)
20 PRINT LEFT$(A$,2);STRING$(3,"!")
```

11. Graphics Reference

physical LCDは320×320 pixelです。virtual coordinateはSCREENで定義し、縦横比を保ってLCD中央へmappingされます。

名称	構文	説明
SCREEN	SCREEN [width,height]	virtual screenを設定。引数なしは640×480
CLS	CLS	graphics screenをclear
COLOR	COLOR palette / COLOR r,g,b	drawing colorを設定
COLORHSV	COLORHSV h,s,v	HSVからdrawing colorを設定
PSET	PSET x,y[,color]	pixelを描く
LINE	LINE x1,y1,x2,y2[,color]	lineを描く
LINE	LINE -x,y[,color]	current graphics pointからlineを描く
CIRCLE	CIRCLE x,y,r[,color]	circleを描く
BOX	BOX x1,y1,x2,y2[,filled[,color]]	rectangleを描く
PAINT	PAINT x,y[,color]	closed regionをflood fill
POINT	POINT(x,y)	pixel query
FLUSH	FLUSH	graphics bufferをLCDへ反映
GLOCATE	GLOCATE x,y	graphics text cursorを設定
GPRINT	GPRINT expression[,expression...]	graphics textを描く
LOADIMAGE	LOADIMAGE filename\$[,x,y]	24-bit BMP読み込み (v0.94 Stage 3)
SAVEIMAGE	SAVEIMAGE filename\$[,x1,y1,x2,y2]	BMP保存・正式構文
SAVE IMAGE	SAVE IMAGE filename\$[,x1,y1,x2,y2]	SAVEIMAGEの旧互換構文

COLORのRGB componentは0～255、COLORHSVのhはdegree、s/vは0～1です。PAINTはfill対象が確定しない場合、または領域が大きい場合に時間を要します。

```

10 SCREEN 320,320:CLS
20 COLOR 255,255,255
30 LINE 0,0,319,319
40 CIRCLE 160,160,80
50 FLUSH

```

12. PCG Reference

GDEF

分類 Graphics Statement 構文 [GDEF "character"="hexdata"](#) / [GDEF "character"=""](#) / [GDEF CLEAR](#) 説明
printable ASCII 0x20~0x7Eに8×8 PCG glyphを定義します。

mono PCGは16 hex digits (8 rows×1 byte) です。bit 7がleft pixelです。set bitはcurrent COLOR、clear bitはbackgroundを描きます。

```

10 GDEF "A"="183C66667E666600"
20 GLOCATE 20,20:GPRINT "A"

```

128 hex digitsでは64個のpalette index byteによるindexed-color PCGです。index 0はtransparentです。

GPALETTE

分類 Graphics Statement 構文 [GPALETTE index,r,g,b](#) / [GPALETTE index,rgb24](#) / [GPALETTE RESET](#) 説明
indexed-color PCGのpaletteを設定します。indexは1~255です。

```

10 GPALETTE 8,18,38,84
20 GPALETTE 9,&H1ED6FF
30 GPALETTE RESET

```

13. Audio Reference

BEEP

分類 Audio Statement 構文 [BEEP frequency_hz,duration_ms](#) 範囲 frequency 20~20,000 Hz、duration 1~60,000 ms。例 [BEEP 880,150](#)

PLAY

分類 Audio Statement 構文

```

PLAY voice1$[,voice2$[,voice3$]]
PLAY PAUSE
PLAY RESUME
PLAY WAIT
PLAY STOP
PLAYING()

```

最大3 voiceです。PLAYはbackgroundで開始します。PLAY WAITは完了まで待機します。PLAY STOPはMML playbackを停止します。

MML

token	意味
Tn	tempo、32～400
On	octave、0～8
Ln	default note length。1、2、4、8、16、32
Vn	level、0～15
C D E F G A B	note
R	rest
# / +	sharp
-	flat
< / >	octave down / up
number	note length
.	dotted note

MMLは1 voiceあたり384 charactersまでです。

```
10 PLAY "T12005L8V12 CDEFGAB>C", "T12004L4V8 CEGC"
20 PLAY WAIT
```

WAV / MP3

名称	構文	説明
WAVPLAY	WAVPLAY "file"	background WAV / MP3 playback開始
WAVPAUSE	WAVPAUSE	playbackをpause
WAVRESUME	WAVRESUME	playbackをresume
WAVSTOP	WAVSTOP	playbackを停止

file内容からformatを判定します。対応formatは次のとおりです。

- RIFF/WAVE PCM (format 1)、8/16-bit、mono/stereo、11,025／22,050／44,100 Hz
- MP3、44,100／48,000 Hz、mono/stereo

compressed WAV、24-bit WAV、48 kHz WAV、対応外sample rateのMP3は対象外です。MP3再生時は必要に応じてCPU 200 MHzへ自動移行し、停止後に元のprofileへ戻ります。

BREAKはPLAY/WAVを停止します。END、STOP、通常のruntime error後もbackground audioは継続します。

14. I2C Reference

I2C SCAN

分類 I2C Statement 構文 [I2C SCAN](#) 説明 7-bit addressをscanします。

I2CREAD

分類 Numeric Function 構文 [I2CREAD\(address,register\)](#) 説明 external I2Cから1 byteをreadします。

I2CWRITE

分類 I2C Statement 構文 `I2CWRITE address,register,value` 範囲 address 0x08~0x77、register 0~255、value 0~255。

```
10 I2C SCAN
20 V=I2CREAD(&H51,&H02)
30 I2CWRITE &H20,&H01,&HFF
```

15. Other BASIC Statements

名称	構文	説明
LOCATE	LOCATE column,row	console text cursor位置を設定
RANDOMIZE	RANDOMIZE [seed]	pseudo-random sequenceを初期化
REM	REM text	comment
'	' text	comment

16. REPL Command Reference

以下はBASIC statementではなく、`BASIC>` promptで実行するREPL commandです。

command	構文	説明
EDIT	EDIT	Full-Screen BASIC Editorを開く
LIST	LIST	current programを表示。BREAK/Esc/Ctrl-Cで途中停止
RUN	RUN	current programを実行。開始時にRUN...を表示
NEW	NEW	program、Direct scalar、current filenameをclear
CLEAR	CLEAR	Direct scalarをclear
CLS	CLS	console/graphics画面をclear
LOAD	LOAD "name"	BASIC sourceをload。開始時にLOADING... (filename)を表示
SAVE	SAVE / SAVE "name"	current fileへsave/Save As
FILES	FILES ["directory"]	SDのBASIC一覧。省略時はroot、指定時は相対directory
DIR	DIR ["directory"]	SDのfile/directory一覧
SD	SD / SD STATUS / SD REMOUNT	SD status/remount
SCREENSHOT	SCREENSHOT ["name"]	LCD BMPをSDへ保存
XRECV	XRECV "name"	XMODEM receive
XSEND	XSEND "name"	XMODEM send
YRECV	YRECV	YMODEM receive
YSEND	YSEND "name"	YMODEM send
DATE	DATE [yyyy-mm-dd]	date表示/設定
TIME	TIME [hh:mm:ss]	time表示/設定
DATETIME	DATETIME	dateとtimeを表示
SERIAL	SERIAL ...	serial route設定表示/変更

command	構文	説明
CONSOLE	CONSOLE LCD ONLY等	console mode変更
PROFILE	PROFILE ...	execution profile表示／制御
STANDBY	STANDBY	RAM保持待機
MENU	MENU	Control Centerを開く
HELP	HELP	command概要

プログラムファイルはpromptのLOAD／SAVEで扱い、画像ファイルはBASICのLOADIMAGE／SAVEIMAGEで扱います。画像命令はprogram内でもpromptでも同じ構文を使えます。SAVE IMAGEは画像保存の旧互換構文です。EDITはprogram内には記述できません。

v0.92 storage / compile behavior

Stored programの初回RUNではsourceをcompileし、PSRAMが利用できる場合はCompiled Program cacheへ保存します。同じsourceを再度RUNするとcacheを復元して内部SRAMのexecution workspaceへ展開します。VMはPSRAM上のopcodeを直接実行しません。sourceを編集、LOAD、NEW、storage mode切替した場合はrevisionが変化し、古いcacheは再利用されません。

Direct scalar stateもPSRAMへ保存できますが、Direct statement実行時は内部SRAM working copyへ復元して処理します。これらは言語仕様を変更せず、runtime memory使用量と再compile overheadを改善する実装です。

17. Runtime Limits and Errors

v0.94では、従来のerror messageを保持し、Compile Error／Runtime Error、ClassicのBASIC行番号またはStructuredのsource row、取得可能な該当source本文を追加表示します。source previewは最大191文字で、省略を明示します。FUNCTION内部では関数名と呼出元row／depthを表示します。信頼できるcolumnと数値error codeは現行engineから取得できないため表示しません。

RUN失敗後もREPLへ戻ります。EDITで変更していないerror rowへ移動し、DIAGNOSTICSでboot session内のLast Errorを確認できます。Last Errorは次の失敗で置換され、成功やBREAKでは残ります。BASICのsource／mode／filenameが変わった場合や保存元がなくなった場合、古い位置へは移動しません。SUBは現行言語では未対応です。GOSUBの呼出履歴は今回のFUNCTION traceに含みません。

resource	limit
compiled operations	1,536
symbols	64
compiled line map	inline 256 + extended map up to program line count
FOR nesting	16
WHILE nesting	16
DO nesting	16
SELECT CASE nesting (Structured)	16
DATA items / program	4,096 (number/string pool制約も適用)
numeric array cells	4,096
string array cells	512
string array element	127 characters
MML per voice	384 characters
INTERNAL program / PSRAM	1,024 lines / 2,047 body chars

resource	limit
INTERNAL program / SRAM fallback	256 lines / 191 body chars
SD program	1,024 lines / 2,047 body chars
Direct / prompt input line	191 body chars

代表的なerror :

category	例	対処
Compile	SYNTAX ERROR、TYPE MISMATCH	syntaxとdata typeを確認
Control flow	NEXT WITHOUT FOR、WEND WITHOUT WHILE	pairを確認
Resource	PROGRAM TOO COMPLEX、TOO MANY VARIABLES	programを分割、variableを削減
Storage	LINE TOO LONG FOR INTERNAL PSRAM、LINE TOO LONG FOR SRAM FALLBACK、PROGRAM TOO LARGE FOR INTERNAL MODE	current backend capacityを確認。PSRAM利用時のINTERNALは1024×2047
I/O	transfer timeout、SD mount failure	route、cable、SD状態を確認

18. Quick Reference

Statements (alphabetical)

v0.93 Stage 3C～3E追加 : [DATA](#)、[READ](#)、[RESTORE](#) (Classic / Structuredのstored program) 、[EXIT FOR](#)、[EXIT DO](#)、[SELECT CASE](#)、[CASE](#)、[CASE ELSE](#)、[END SELECT](#) (Structured専用) 。

[BEEP](#)、[BOX](#)、[CIRCLE](#)、[CLS](#)、[COLOR](#)、[COLORHSV](#)、[DIM](#)、[DO](#)、[END](#)、[FLUSH](#)、[FOR](#)、[GDEF](#)、[GLOCATE](#)、[GOSUB](#)、[GOTO](#)、[GPALETTE](#)、[GPRINT](#)、[I2C SCAN](#)、[I2CWRITE](#)、[IF](#)、[INPUT](#)、[LET](#)、[LINE](#)、[LOADIMAGE](#)、[LOCATE](#)、[LOOP](#)、[NEXT](#)、[ON ... GOSUB](#)、[ON ... GOTO](#)、[PAINT](#)、[PAUSE](#)、[PLAY](#)、[PRINT](#)、[PSET](#)、[RANDOMIZE](#)、[REM](#)、[RETURN](#)、[SAVE IMAGE](#)、[SAVEIMAGE](#)、[SCREEN](#)、[SLEEP](#)、[STOP](#)、[WAVPAUSE](#)、[WAVPLAY](#)、[WAVRESUME](#)、[WAVSTOP](#)、[WEND](#)、[WHILE](#)

Functions (alphabetical)

[ABS](#)、[ACOS](#)、[ASC](#)、[ASIN](#)、[ATAN2](#)、[ATN](#)、[CHR\\$](#)、[CLAMP](#)、[COS](#)、[DEG](#)、[EXP](#)、[I2CREAD](#)、[INKEY](#)、[INSTR](#)、[INT](#)、[LEFT\\$](#)、[LEN](#)、[LN](#)、[LOG](#)、[MAX](#)、[MID\\$](#)、[MIN](#)、[PI](#)、[PLAYING](#)、[POINT](#)、[RAD](#)、[RND](#)、[RNDI](#)、[RIGHT\\$](#)、[SGN](#)、[SIN](#)、[SPC](#)、[SQR](#)、[STR\\$](#)、[STRING\\$](#)、[TRIM\\$](#)、[LTRIM\\$](#)、[RTRIM\\$](#)、[REPLACE\\$](#)、[SPACE\\$](#)、[DATE\\$](#)、[TIME\\$](#)、[TAB](#)、[TAN](#)、[TIMER](#)、[VAL](#)

実行可能なより大きいsampleはrepositoryの [examples/](#) を参照してください。

19. Structured BASIC

LOADは空行を除く全行に番号があればClassic、全行に番号がなければStructuredと判定します。混在はMIXED SOURCE MODEで拒否し、現在sourceを保持します。空／空行だけのfileは現在sessionのmodeを保持します。Structured保存は行番号を付けず、空行・indent・本文を保持します（改行はLF）。LISTも番号なしです。StructuredではGOTO、GOSUB、ON GOTO/GOSUBを使用できません。Direct mode、Classicの番号入力／分岐／RETURNは従来どおりです。

THEN直後が行末なら複数行IFです。statementが続けば従来のsingle-line IFです。

```

SCORE=75
IF SCORE>=100 THEN
    PRINT "CLEAR"
ELSEIF SCORE>=50 THEN
    PRINT "GOOD"
ELSE
    PRINT "TRY AGAIN"
END IF

```

ELSEIFは複数、ELSEは省略可能です。IFは入れ子にでき、FOR/NEXT、WHILE/WEND、DO/LOOP、DO WHILE/LOOP、DO/LOOP UNTILと組み合わせられます。blockの欠落／二重ELSE／ELSE後ELSEIF／loop境界の交差はcompile errorです。keywordは大文字小文字を区別せず、indentは任意です。

EXIT FOR / EXIT DO (Structured専用)

EXIT FORは実行中の最内側FORから対応するNEXTの直後へ移ります。**EXIT DO**は最内側DOから対応するLOOPの直後へ移ります。IFやSELECT CASE内、FUNCTION内でも利用できます。EXIT対象のloopがない場合は**EXIT FOR WITHOUT FOR**／**EXIT DO WITHOUT DO**です。EXIT WHILE、EXIT FUNCTIONは未対応です。

```

DO
    FOR I=1 TO 10
        IF I=3 THEN EXIT DO
    NEXT I
LOOP
FOR J=1 TO 3
    PRINT J
NEXT J

```

EXIT DOは対象DOの内側で作られたFOR frameをすべて破棄します。FUNCTION内から呼出し元のFOR frameを終了することはできません。上の例ではI=3でDOを抜け、後続のFORが1、2、3を表示します。Classicは従来のGOTO等を使用してください。

SELECT CASE / CASE / CASE ELSE / END SELECT (Structured専用)

SELECT CASE expressionの選択式は数値または文字列です。開始時に一度だけ評価し、最初に一致したCASEのbodyだけを実行します。body終了後はEND SELECTの直後へ進み、次のCASEへのfallthroughはありません。

```

SELECT CASE SCORE
CASE 1, 2, 3
    PRINT "LOW"
CASE 4 TO 10, 20
    PRINT "HIGH"
CASE ELSE
    PRINT "OTHER"
END SELECT

```

CASEにはliteralのみ指定できます。数値のdecimal、符号、指数表記、&H、引用符付き文字列を使用でき、commaで候補を並べられます。数値rangeは**low TO high**で両端を含み、exact値と混在できます。low>highのrangeは一致しません。文字列range、CASE IS、比較演算子による条件、variableやfunction呼出しを含むCASE式は未対応です。選択式とCASEの型は一致させてください。

CASE ELSEは省略可能で、最大1回・最後に置きます。CASE ELSEがなく、どれにも一致しなければEND SELECTの後へ進みます。少なくとも1つのCASEが必要です。SELECT CASEは16段まで入れ子にでき、IF／FOR／WHILE／DO／FUNCTIONと組み合わせられます。CASE・END SELECTによる他block境界の交差はcompile errorです。

Full-Screen EditorのAlt+MはSELECT CASEとEND SELECTを往復し、CASE／CASE ELSEからそのSELECT CASEへ移動します。文字列やREMのkeyword、SELECTORやCASEVALUEなどidentifierの一部は対象になりません。

FUNCTION / END FUNCTION / RETURN

```

PRINT SQUARE(12)
FUNCTION SQUARE(X)
    RETURN X*X
END FUNCTION

FUNCTION WRAP$(S$)
    RETURN "["+S$+"]"
END FUNCTION
PRINT WRAP$("CPB")

```

FUNCTIONはStructured top-levelだけに置けます。定義より前から呼べます。definition bodyはmain実行で飛び越します。名前末尾\$は文字列返値、その他は数値返値です。parameterも同じ型規則で値渡しです。括弧はzero-argument呼出しにも必要です。argument数／型とRETURN expressionの型はcompile時に検査します。

数値localの初期値は0、文字列localは空です。parameter、代入先、未宣言の参照は関数localとなり、関数ごと／呼出しごとに独立します。同名globalを暗黙に参照しません。

```

SCORE=0
FUNCTION ADD_SCORE(POINT)
    GLOBAL SCORE
    SCORE=SCORE+POINT
    RETURN SCORE
END FUNCTION
PRINT ADD_SCORE(10)
PRINT ADD_SCORE(20)
PRINT SCORE

```

GLOBALはFUNCTION body先頭の実行statementより前に宣言し、read/write両方へ適用します。複数名はGLOBAL SCORE,NAME\$のようにcommaで区切ります。commentと空行は実行statementではありません。parameter/localとGLOBALの同名は拒否します。関数内arrayは非対応です。

RETURN expressionはFUNCTION内専用です。数値／文字列の型は一致させてください。RETURNなし、expressionなしはcompile errorです。RETURNを通らずEND FUNCTIONに到達するとFUNCTION RETURN MISSINGになります。Classic RETURNはGOSUBから戻る既存動作を維持します。

関数名はcase-insensitiveで、FとF\$は同一base nameです。duplicate、builtinとのbase-name衝突、同base-name scalarとの曖昧性は拒否します。Structured identifierは最大16文字です。

制限	値
FUNCTION数	32
parameter数／関数	8
scalar local数／関数（parameterを含む）	128
nested call／recursion depth	16
string local／parameter／return	127文字
global symbol数	64

再帰を利用できます。17段目はFUNCTION CALL DEPTH、SRAM不足はOUT OF MEMORYとなります。大きいstring frameでは16段より前にSRAM不足となる場合があります。BREAK／runtime error／通常return後にframeは解放されます。

```

FUNCTION FACT(N)
  IF N<=1 THEN
    RETURN 1
  END IF
  RETURN N*FACT(N-1)
END FUNCTION
PRINT FACT(5)

```

期待値は120です。Structured errorはAT ROWを表示し、関数内ではIN FUNCTION、CALLED FROM ROWとDEPTH也表示します。ROWはsource中の位置であり、BASICの分岐行番号ではありません。Classic errorのIN 行番号は従来どおりです。

20. 実装上限と非対応構文

項目	v0.93の制限
IL命令	1,536 slots
literal string pool	6,144 bytes
global symbol	64
identifier	最大16文字
source容量	PSRAM INTERNAL／SDは1024×2047、SRAM fallbackは256×191
prompt／Direct入力	191文字

ソース容量内でもIL／pool上限に達する場合があります。BAS保存はLF改行です。パスはSDルート相対で区切りを含め79文字です。PSRAMなしではEditor履歴を利用できません。

FUNCTION内のlocal配列、GLOBAL配列アクセス、SUB、labels、STATIC、optional parameter、overload、moduleは非対応です。INPUT強化、BASIC File I/O、TRACE／Debuggerは将来候補です。OTAとBASICネイティブコンパイラは含みません。

21. DATA / READ / RESTORE (Classic / Structured)

stored program内で、program全体に1本のDATA streamを定義します。DATA itemはdecimal、符号付き数値、指数表記、&Hによる数値literal、引用符付き文字列literalに限定します。DATA A+1、DATA SIN(1)、DATA PI、引用符のないDATA HELLOはcompile errorです。空文字列""、引用符内のcommaやspaceを利用できます。文字列の引用符・数値書式は既存literalと同じです。

```

10 DATA -1,2.5,1.0E3,&H10,"A,B",""
20 READ A,B,C,D,$$,T$
30 PRINT A;B;C;D;$$;T$
40 RESTORE
50 READ A
60 PRINT A

```

ClassicではBASIC行番号順、Structuredでは論理source row順に全DATAを収集します。DATAには実行時opcodeがなく、IFの不成立やloopの未実行に影響されません。StructuredのDATAはprogram scopeに限り、FUNCTION内部には書けません。program scopeのIF／loop内に置かれた定義もsource順に収集します。

READ A,B,C\$は順にitemを読み、数値variableには数値、文字列variableには文字列だけを代入します。暗黙変換はしません。型不一致はREAD TYPE MISMATCH、item不足はOUT OF DATAです。既存DIMした1次元／2次元配列へのREAD A(I),B\$(I,J)を利用できます。FUNCTIONではscalar local、GLOBAL宣言済みscalarへREADできますが、FUNCTION内配列は既存仕様どおり非対応です。cursorは関数間でもprogram全体で共有します。

RESTOREは引数なしで先頭へ戻します。RESTORE 1000、label、function指定はsyntax errorです。RUNごとに先頭から開始し、repeated RUN、cache-hit RUN、BREAK/runtime error後のRUNでも同じです。READ成功時だけcursorを進め、実行時cursorはcacheへ保存しません。

DATA/READ/RESTOREはDirect modeではDATA/READ/RESTORE REQUIRE STORED PROGRAMです。前回RUNしたstreamへの接続やcursorの共有は行いません。DATA item上限は4096で、既存number pool 1536/string pool 6144 bytesの上限とSRAM容量も適用されます。tableは1 itemあたり2 bytesのdynamic allocationで、DATAなしなら0 bytesです。

v0.93は引き続きVMでBASICを実行します。FUNCTION有無に応じたVM内部C++実装の選択と融合命令で処理を改善しています。single precisionと丸め規則を維持し、新local数値helperではFMA contractionを禁止しています。PROFILE COUNTは機構確認用、通常速度はPROFILE OFFで測定してください。RUNの表示時間はVM実行で、compile-onlyやcache-onlyの時間を示しません。

22. Statement / Function / REPL 個別リファレンス

この章は、実装から確認したv0.93の言語要素を、検索しやすい独立entryとしてまとめたものです。各枠の「対応」はstored programで利用できることを示します。REPL commandの「共通」は、どちらのprogram modeを編集集中でもBASIC> promptから利用できることを示します。Direct modeの例外は本文に記します。

Assignment / LET

Classic 対応

Structured 対応

分類: 代入statement。構文: [LET] variable=expression。scalarまたはarray elementへ値を保存します。

```
10 TEMP=23.5
20 LET LABEL$="ROOM"
30 PRINT LABEL$;"=";TEMP
```

期待結果はROOM=23.5です。Direct modeでも利用できます。

PRINT

Classic 対応

Structured 対応

分類: 出力statement。構文: PRINT [expression[;], expression]...。semicolonは連結、commaは次のfieldへ進みます。

```
10 TEMP=23.5
20 PRINT "TEMPERATURE=";TEMP;" C"
```

期待結果はTEMPERATURE=23.5 Cです。Direct modeでも利用できます。

INPUT

Classic 対応

Structured 対応

分類: 入力statement。構文: INPUT ["prompt";] variable。scalarを1個入力します。

```
INPUT "TARGET TEMP";T
PRINT "SET=";T
```

Structured例です。入力した値がSET=に続いて表示されます。Direct modeでも利用できます。

DIM

Classic 対応

Structured 対応

分類: 配列statement。構文: DIM name(size[,size])[,name(...)...]。1Dまたは2D配列を確保します。

```
10 DIM SAMPLE(9)
20 SAMPLE(0)=42
30 PRINT SAMPLE(0)
```

期待結果は42です。FUNCTION内の配列は非対応です。

REM / apostrophe comment

Classic 対応

Structured 対応

分類: comment。構文: **REM text**または'**text**。

```
10 REM SENSOR WARM-UP
20 ' WAIT 100 MS
30 SLEEP 100
```

commentは実行されません。文字列内の**REM**やapostropheはcomment開始として扱いません。

Single-line IF

Classic 対応

Structured 対応

分類: 条件statement。構文: **IF condition THEN statement-or-line [ELSE statement-or-line]**。

```
10 A=3
20 IF A>0 THEN PRINT "PLUS" ELSE PRINT "NOT PLUS"
```

期待結果は**PLUS**です。StructuredでもstatementがTHENの後に続く場合はsingle-line形式です。Structuredからline numberへ分岐はできません。

Block IF / ELSEIF / ELSE / END IF

Classic 非対応

Structured 対応

分類: Structured条件block。THEN直後を行末にし、必要に応じて**ELSEIF**と**ELSE**を置きます。

```
SCORE=75
IF SCORE>=80 THEN
  PRINT "A"
ELSEIF SCORE>=60 THEN
  PRINT "B"
ELSE
  PRINT "C"
END IF
```

期待結果は**B**です。Classicではsingle-line IFを使用します。

FOR / NEXT

Classic 対応

Structured 対応

分類: count loop。構文: **FOR variable=start TO limit [STEP increment] / NEXT [variable]**。

```
FOR I=1 TO 5 STEP 2
  PRINT I
NEXT I
```

期待結果は1、3、5です。Classicでは各行に行番号を付けます。nesting上限は16です。

WHILE / WEND

Classic 対応

Structured 対応

分類: 前判定loop。構文: **WHILE condition / WEND**。

```
N=1
WHILE N<=3
  PRINT N
  N=N+1
WEND
```

期待結果は1、2、3です。

DO / LOOP

Classic 対応

Structured 対応

分類: loop。構文: **DO / LOOP [UNTIL condition]**。

```

DO
    K=INKEY
LOOP UNTIL K=27
PRINT "ESC"

```

Escape code 27を受け取るとloopを抜けます。PicoCalcの実際のEscape codeはkey-code表も確認してください。

EXIT FOR Classic 非対応 Structured 対応

分類: loop脱出。構文: **EXIT FOR**。最内側FORの直後へ進みます。

```

FOR I=1 TO 10
    IF I=4 THEN EXIT FOR
NEXT I
PRINT "STOP=";I

```

期待結果は**STOP=4**です。Classicでは条件分岐とline branchを使用します。

EXIT DO Classic 非対応 Structured 対応

分類: loop脱出。構文: **EXIT DO**。最内側DOの直後へ進み、そのDO内で作られた内側FOR frameも安全に破棄します。

```

DO
    FOR I=1 TO 10
        IF I=3 THEN EXIT DO
    NEXT I
LOOP
FOR J=1 TO 3
    PRINT J
NEXT J

```

期待結果は1、2、3です。後続FORが正常に動くことが利用者から見たcleanupの効果です。

GOTO Classic 対応 Structured 非対応

分類: line branch。構文: **GOTO line**。

```

10 PRINT "START"
20 GOTO 40
30 PRINT "SKIP"
40 PRINT "END"

```

STARTと**END**を表示します。Structuredで**GOTO**はcompile errorです。

GOSUB Classic 対応 Structured 非対応

分類: line subroutine。構文: **GOSUB line**。

```

10 GOSUB 100
20 END
100 PRINT "SUB"
110 RETURN

```

期待結果は**SUB**です。StructuredではFUNCTIONを使用します。

RETURN - Classic GOSUB return Classic 対応 Structured 非対応

分類: subroutine復帰。構文: **RETURN**。直近のClassic **GOSUB**呼出元へ戻ります。

```
10 GOSUB 100
20 PRINT "BACK":END
100 PRINT "CALL"
110 RETURN
```

CALL、**BACK**の順に表示します。StructuredのRETURN expressionとは別の構文です。

ON ... GOTO

Classic 対応

Structured 非対応

分類: selector branch。構文: **ON expression GOTO line[,line...]**。selectorは1始まりです。

```
10 N=2
20 ON N GOTO 100,200,300
100 PRINT "ONE":END
200 PRINT "TWO":END
300 PRINT "THREE":END
```

期待結果は**TWO**です。

ON ... GOSUB

Classic 対応

Structured 非対応

分類: selector subroutine。構文: **ON expression GOSUB line[,line...]**。

```
10 N=1
20 ON N GOSUB 100,200
30 PRINT "DONE":END
100 PRINT "A":RETURN
200 PRINT "B":RETURN
```

期待結果は**A**、**DONE**です。

SELECT CASE

Classic 非対応

Structured 対応

分類: 複数分岐block。構文: **SELECT CASE expression**。

```
CHOICE=4
SELECT CASE CHOICE
CASE 1
    PRINT "START"
CASE 2,3
    PRINT "SETTINGS"
CASE 4 TO 9
    PRINT "SENSOR RANGE"
CASE ELSE
    PRINT "UNKNOWN"
END SELECT
```

期待結果は**SENSOR RANGE**です。selectorは1回だけ評価されます。

CASE

Classic 非対応

Structured 対応

分類: SELECT条件。構文: **CASE literal[,literal|low TO high...]**。

```
SELECT CASE CODE
CASE 1,2,3
    PRINT "LOW"
CASE 10 TO 20
    PRINT "MID"
END SELECT
```

値listと両端を含むrangeを利用できます。variable、function call、**CASE IS**、文字列rangeは非対応です。

CASE ELSE

Classic 非対応

Structured 対応

分類: SELECT既定分岐。どのCASEにも一致しない場合に実行します。

```
SELECT CASE MODE
CASE 1
    PRINT "AUTO"
CASE ELSE
    PRINT "MANUAL"
END SELECT
```

MODEが1以外ならMANUALです。1つだけ、最後に置きます。

END SELECT

Classic 非対応

Structured 対応

分類: SELECT終端。

```
SELECT CASE 1
CASE 1
    PRINT "OK"
END SELECT
```

期待結果はOKです。対応するSELECTがない場合はcompile errorです。

FUNCTION

Classic 非対応

Structured 対応

分類: user function定義。構文: **FUNCTION** name([parameters])。数値functionは通常名、文字列functionは末尾\$です。

```
PRINT SQUARE(12)
FUNCTION SQUARE(X)
    RETURN X*X
END FUNCTION
```

期待結果は144です。定義前から呼べ、再帰も利用できます。

END FUNCTION

Classic 非対応

Structured 対応

分類: FUNCTION終端。

```
PRINT WRAP$("CPB")
FUNCTION WRAP$(S$)
    RETURN "["+S$+"]"
END FUNCTION
```

期待結果は[CPB]です。FUNCTIONを入れ子にはできません。

GLOBAL

Classic 非対応

Structured 対応

分類: FUNCTION内global宣言。構文: **GLOBAL** name[,name...]. 実行statementより前に置きます。

```
TOTAL=10
PRINT ADD(5)
FUNCTION ADD(X)
    GLOBAL TOTAL
    TOTAL=TOTAL+X
    RETURN TOTAL
END FUNCTION
```

期待結果は15です。FUNCTION外、local／parameterとの同名、array accessは非対応です。

RETURN expression - Structured FUNCTION return

Classic 非対応

Structured 対応

分類: FUNCTION返値。構文: RETURN expression。

```
PRINT FACT(5)
FUNCTION FACT(N)
  IF N<=1 THEN RETURN 1
  RETURN N*FACT(N-1)
END FUNCTION
```

期待結果は120です。型はFUNCTION名の数値／文字列型と一致させます。Classicの引数なしRETURNとは別です。

DATA

Classic 対応

Structured 対応

分類: literal data。構文: DATA literal[,literal...]。Direct modeは非対応です。

```
10 DATA 10,20,"READY"
20 READ A,B,S$
30 PRINT A+B;" ";S$
```

期待結果は30 READYです。DATA A+1のようなexpressionは使えません。

READ

Classic 対応

Structured 対応

分類: DATA読み出し。構文: READ variable[,variable...]。Direct modeは非対応です。

```
DATA 2,4,6
FOR I=1 TO 3
  READ V
  PRINT V
NEXT I
```

期待結果は2、4、6です。型不一致はREAD TYPE MISMATCH、不足はOUT OF DATAです。

RESTORE

Classic 対応

Structured 対応

分類: DATA cursor reset。構文: RESTORE。Direct modeは非対応です。

```
DATA 7,8
READ A
RESTORE
READ B
PRINT A;B
```

期待結果は7、7です。RESTORE 100などline指定は非対応です。

END

Classic 対応

Structured 対応

分類: program終了。

```
10 PRINT "DONE"
20 END
30 PRINT "NO"
```

DONEだけを表示します。background audioは自動停止しません。

STOP

Classic 対応

Structured 対応

分類: program停止。v0.93ではENDと同じHALTとして扱います。

```
PRINT "BEFORE"
STOP
PRINT "AFTER"
```

BEFOREだけを表示します。BREAKとは異なりbackground audioを自動停止しません。

SLEEP

Classic 対応

Structured 対応

分類: timing statement。構文: **SLEEP** milliseconds。

```
10 PRINT "WAIT"
20 SLEEP 500
30 PRINT "GO"
```

約0.5秒待ちます。待機中もbackground serviceとBREAK受付を継続します。

PAUSE

Classic 対応

Structured 対応

分類: key wait statement。

```
PRINT "PRESS A KEY"
PAUSE
PRINT "CONTINUE"
```

通常keyを受け取るまで画面を保ちます。

RANDOMIZE

Classic 対応

Structured 対応

分類: random初期化。構文: **RANDOMIZE** [seed]。

```
10 RANDOMIZE 123
20 PRINT RNDI(10)
```

同じseedは再現可能なpseudo-random sequenceの確認に使えます。

LOCATE

Classic 対応

Structured 対応

分類: console cursor。構文: **LOCATE** column,row。

```
10 LOCATE 5,3
20 PRINT "STATUS"
```

text cursorを指定位置へ移して表示します。

SCREEN

Classic 対応

Structured 対応

分類: graphics設定。構文: **SCREEN** [width,height]。引数なしは640×480 virtual coordinateです。

```
10 SCREEN 320,320
20 CLS
```

320×320 virtual screenを設定し、画面を消去します。

CLS statement

Classic 対応

Structured 対応

分類: graphics clear。

```
10 COLOR 255,255,255
20 PSET 10,10
30 CLS
```

graphics screenをclearします。BASIC>のREPL CLSも画面clearですが、entry pointが異なります。

COLOR

Classic 対応

Structured 対応

分類: drawing color。構文: COLOR paletteまたはCOLOR r,g,b。

```
10 COLOR 255,0,0
20 LINE 0,0,100,0
```

赤いlineを描きます。componentは0~255です。

COLORHSV

Classic 対応

Structured 対応

分類: HSV drawing color。構文: COLORHSV h,s,v。

```
10 COLORHSV 120,1,1
20 CIRCLE 80,80,30
```

緑系のcircleを描きます。hはdegree、s/vは0~1です。

PSET

Classic 対応

Structured 対応

分類: pixel描画。構文: PSET x,y[,color]。

```
10 SCREEN 320,320
20 PSET 160,160
30 FLUSH
```

画面中央に1 pixelを描いて反映します。

LINE

Classic 対応

Structured 対応

分類: line描画。構文: LINE x1,y1,x2,y2[,color]またはLINE -x,y[,color]。

```
10 LINE 10,10,100,10
20 LINE -100,100
```

2本目はcurrent graphics pointから描きます。

CIRCLE

Classic 対応

Structured 対応

分類: circle描画。構文: CIRCLE x,y,r[,color]。

```
10 CIRCLE 160,160,60
20 FLUSH
```

中央に半径60のcircleを描きます。

BOX

Classic 対応

Structured 対応

分類: rectangle描画。構文: BOX x1,y1,x2,y2[,filled[,color]]。

```
10 BOX 20,20,120,80,1
20 FLUSH
```

filled rectangleを描きます。

PAINT

Classic 対応

Structured 対応

分類: flood fill。構文: PAINT x,y[,color]。

```
10 CIRCLE 100,100,40
20 PAINT 100,100
30 FLUSH
```

閉じたcircle内部を塗ります。広い領域では時間がかかる場合があります。

FLUSH

Classic 対応

Structured 対応

分類: graphics反映。

```
10 PSET 1,1
20 FLUSH
```

buffered graphicsをLCDへ反映します。

GLOCATE

Classic 対応

Structured 対応

分類: graphics text cursor。構文: [GLOCATE x,y](#)。

```
10 GLOCATE 20,30
20 GPRINT "TEMP 23.5C"
```

graphics画面の指定位置へ文字を描きます。

GPRINT

Classic 対応

Structured 対応

分類: graphics text。構文: [GPRINT expression\[,expression...\]](#)。

```
10 GLOCATE 8,8
20 GPRINT "X=",42
```

graphics fontでX=と42を描きます。

GDEF

Classic 対応

Structured 対応

分類: PCG定義。構文: [GDEF "character"="hexdata"](#)、空文字で解除、[GDEF CLEAR](#)で全解除。

```
10 GDEF "A"="183C66667E666600"
20 GLOCATE 20,20:GPRINT "A"
```

printable ASCIIに8×8 mono glyphを定義して描きます。

GPALETTE

Classic 対応

Structured 対応

分類: PCG palette。構文: [GPALETTE index,r,g,b](#)、[GPALETTE index,rgb24](#)、[GPALETTE RESET](#)。

```
10 GPALETTE 8,18,38,84
20 GPALETTE 9,&H1ED6FF
```

indexed-color PCG用paletteを設定します。indexは1～255です。

LOADIMAGE

Classic 対応

Structured 対応

分類: 画像用BASIC statement (v0.94 Stage 3)。

Syntax: [LOADIMAGE filename\\$\[,x,y\]](#)

Parameters: [filename\\$](#)はファイル名を返す文字列式です。[x,y](#)は画像の左上の物理LCD座標です。両方省略した場合は0,0です。片方だけの指定はエラーです。有限で32-bit signed integerの範囲にある数値を指定し、小数は0方向に切り捨てます。

Description: SD CardのBMP画像を等倍で描画します。ファイル名にはA\$+".BMP"等の文字列式を使えます。描画色COLORは読み込み後に復元します。POINTの画素状態にも反映します。

Supported image format: 24-bit uncompressed BMP (BI_RGB、planes=1、DIB header=40/108/124 bytes)。BGR色順、4-byte row alignment、bottom-upとtop-downに対応します。JPEG、PNG、16/32-bit、palette、compressionは非対応です。

Coordinate behavior: 320×320 LCDの範囲からはみ出す部分をclippingします。負の座標も許可します。全体が画面外なら描画しませんが、formatとfile sizeは検証します。SCREENの仮想座標変換・拡大縮小は画像I/Oには適用しません。SAVEIMAGEの保存領域と同じ物理pixelです。

File paths: 画像I/Oは共通FilesystemのSD root基準です。images/PHOTO.BMPと/ images/PHOTO.BMPは同じfileです。Filesで表示中のdirectory、loaded programのdirectoryを暗黙のcurrent directoryとして使いません。path全体は拡張子込み79文字まで、空白を含む名前も利用できます。./../、重複separator、backslashは共通ルールで拒否します。.bmp等の大文字小文字を問わず拡張子を認識し、末尾が.BMP以外なら従来どおり.BMPを補完します。PHOTO.PNGはPHOTO.PNG.BMPとなるため、未対応形式の検査では実際に開くfile名に注意してください。

Internal storageはprogram用RAM領域であり、画像fileの保存先にはなりません。

Error behavior: missing fileはIMAGE FILE NOT FOUND、短いheaderはINVALID BMP HEADER、signature不一致はUNSUPPORTED IMAGE FORMAT、非対応BMPはUNSUPPORTED BMP FORMAT、dimension/offset/size不正はINVALID BMP SIZE、payload不足はTRUNCATED BMP、seek/read失敗はIMAGE READ ERRORです。argumentの不足/型違いはLOADIMAGE: ARGUMENT COUNT/FILENAME REQUIRED/BAD COORDINATEです。SD未使用・USB Host所有・Storage busyは既存storage errorを返します。headerと全payloadの長さを最初の描画前に検証します。実際の読込中にSDが外れた場合等は、すでに描画した部分が残る場合があります。

Example (Structured) :

```
CLS
LOADIMAGE "PHOTO.BMP", 0, 0
```

Example (Classic) :

```
10 CLS
20 LOADIMAGE "TEST.BMP", 0, 0
30 SAVEIMAGE "COPY.BMP"
40 END
```

SAVEIMAGE / SAVE IMAGE

Classic 対応

Structured 対応

分類: 画像用BASIC statement。

Syntax: SAVEIMAGE filename\$[,x1,y1,x2,y2]

Parameters: filename\$は文字列式です。省略時は物理LCDの0,0~319,319を保存します。領域を指定する場合は4座標をすべて指定します。両端を含み、逆順の座標は入れ替え、画面外部分はclippingします。交差しない領域はBAD SCREENSHOT REGIONです。座標は有限32-bit signed integerの数値で、小数は0方向へ切り捨てます。

Description: 現在のLCDの表示内容 (text/statusを含む) をSDへ24-bit uncompressed、bottom-up BMPで保存します。既存fileは上書きします。filenameの補完、SD root基準のpath、storage ownershipはLOADIMAGEと同じです。空白・directory・先頭/のpathは共通Filesystemで扱います。

Compatibility: SAVEIMAGEが正式・推奨構文です。SAVE IMAGEは旧互換構文として引き続き使え、compile時に同じstatement (GSAVE) へ正規化します。保存本体は共通で、実行時warningはありません。program保存はREPLのSAVEを使います。

Error behavior: 引数の数、filename型、領域の型/範囲の異常はSAVEIMAGE: ARGUMENT COUNT/FILENAME REQUIRED/BAD REGIONです。file open失敗はCANNOT OPEN FILE、書込/LCD読出し/close失敗は

SCREENSHOT WRITE ERRORです。失敗した場合は不完全なfileが残る場合があります（従来の上書き動作）。

Example (Structured / round-trip) :

```
SCREEN 320,320
CLS
COLOR 255,64,0
BOX 20,20,120,80,1
SAVEIMAGE "SCREEN.BMP"
CLS
LOADIMAGE "SCREEN.BMP",0,0
PAUSE
```

Example (Classic / compatibility) :

```
10 SCREEN 320,320
20 CIRCLE 160,160,80
30 SAVE IMAGE "CIRCLE"
40 CLS
50 LOADIMAGE "CIRCLE.BMP",0,0
60 PAUSE
70 END
```

実機での一括確認sampleは[examples/v094/image-roundtrip-structured.bas](#)と[examples/v094/image-roundtrip-classic.bas](#)です。

BEEP

Classic 対応

Structured 対応

分類: tone出力。構文: **BEEP** frequency_hz,duration_ms。

```
10 BEEP 880,150
```

880 Hzを約150 ms再生します。音量設定と実機speakerにより聞こえ方は異なります。

PLAY

Classic 対応

Structured 対応

分類: MML playback。構文: **PLAY** voice1\$[,voice2\$[,voice3\$]]。

```
10 PLAY "T12005L8 CDEFG"
20 PLAY WAIT
```

短いmelodyを開始して完了を待ちます。**PLAY PAUSE**、**PLAY RESUME**、**PLAY STOP**も使用できます。

WAVPLAY

Classic 対応

Structured 対応

分類: audio file playback。構文: **WAVPLAY** "file"。

```
10 WAVPLAY "ALERT.WAV"
20 PRINT "PLAYING"
```

WAVまたはMP3をbackground再生します。file、SD、format、音量により結果は異なります。

WAVPAUSE

Classic 対応

Structured 対応

分類: audio pause。

```
10 WAVPLAY "MUSIC.MP3"
20 SLEEP 1000
30 WAVPAUSE
```

再生中fileをpauseします。対象fileがない場合の状態は実行環境に依存します。

WAVRESUME

Classic 対応

Structured 対応

分類: audio resume。

```
10 WAVPLAY "MUSIC.MP3"
20 WAVPAUSE
30 WAVRESUME
```

pauseしたfile再生を再開します。

WAVSTOP

Classic 対応

Structured 対応

分類: audio stop。

```
10 WAVPLAY "MUSIC.MP3"
20 SLEEP 500
30 WAVSTOP
```

file playbackを停止します。

I2C SCAN

Classic 対応

Structured 対応

分類: I2C statement。外部I2C busの7-bit addressをscanします。

```
10 I2C SCAN
```

検出addressがconsoleへ表示されます。接続device、配線、電源により結果は異なります。

I2CWRITE

Classic 対応

Structured 対応

分類: I2C write。構文: `I2CWRITE address,register,value`。

```
10 I2CWRITE &H20,&H01,&HFF
```

1 byteを書きます。addressは0x08~0x77、register/valueは0~255です。対象deviceのdatasheetを確認してください。

Operators \, <<, >>, XOR

Classic 対応

Structured 対応

分類: int32 semanticsを使うoperator。結果はsingle-precision floatへ戻ります。

```
10 PRINT 17\5
20 PRINT 1<<4
30 PRINT -9>>1
40 PRINT 5 XOR 3
```

期待結果は3、16、-5、6です。真のINTEGER型やA%はなく、 2^{24} を超える整数は低位bitを保持できない場合があります。Direct modeでも利用できます。

ABS

Classic 対応

Structured 対応

分類: numeric function。構文: `ABS(x)`。絶対値を返します。

```
10 PRINT ABS(-12.5)
```

期待結果は12.5です。

INT

Classic 対応

Structured 対応

分類: numeric function。構文: `INT(x)`。実装の整数方向へ変換します。

```
10 PRINT INT(3.9)
```

期待結果は整数化された3です。

VAL

Classic 対応

Structured 対応

分類: conversion function。構文: VAL(string\$)。

```
10 A=VAL("12.5")
20 PRINT A+1
```

期待結果は13.5です。

STR\$

Classic 対応

Structured 対応

分類: conversion function。構文: STR\$(number)。

```
10 S$="V="+STR$(42)
20 PRINT S$
```

数値を文字列へ変換してV=42を表示します。

LEN

Classic 対応

Structured 対応

分類: string function。構文: LEN(string\$)。

```
10 PRINT LEN("PICOALC")
```

期待結果は8です。

CHR\$

Classic 対応

Structured 対応

分類: string function。構文: CHR\$(code)。

```
10 PRINT CHR$(65)
```

期待結果はAです。

ASC

Classic 対応

Structured 対応

分類: numeric string function。構文: ASC(string\$)。

```
10 PRINT ASC("A")
```

期待結果は65です。

LEFT\$

Classic 対応

Structured 対応

分類: string function。構文: LEFT\$(string\$,length)。

```
10 PRINT LEFT$("PICOALC",4)
```

期待結果はPICOです。

RIGHT\$

Classic 対応

Structured 対応

分類: string function。構文: RIGHT\$(string\$,length)。

```
10 PRINT RIGHT$("PICOALC",4)
```

期待結果はCALCです。

MID\$

Classic 対応

Structured 対応

分類: string function。構文: `MID$(string$,start[,length])`。

```
10 PRINT MID$("PICO CALC",5,4)
```

期待結果は**CALC**です。

INSTR

Classic 対応

Structured 対応

分類: string search。構文: `INSTR(string$,find$)`。

```
10 PRINT INSTR("SENSOR=OK","OK")
```

部分文字列の位置を返します。見つからない場合は0です。

STRING\$

Classic 対応

Structured 対応

分類: string repeat。構文: `STRING$(count,string$)`。

```
10 PRINT STRING$(5,"-")
```

期待結果は**-----**です。

SPC

Classic 対応

Structured 対応

分類: spacing function。構文: `SPC(count)`。

```
10 PRINT "A";SPC(3);"B"
```

AとBの間へ3 spaceを入れます。

TAB

Classic 対応

Structured 対応

分類: PRINT用tab function。構文: `TAB(column)`。

```
10 PRINT "ID";TAB(10);"VALUE"
```

VALUEを指定columnへ配置します。

RND

Classic 対応

Structured 対応

分類: pseudo-random function。構文: `RND`または`RND()`。

```
10 RANDOMIZE 123
20 PRINT RND
```

pseudo-random numberを返します。再現性が必要なら先にseedを指定します。

RNDI

Classic 対応

Structured 対応

分類: integer random function。構文: `RNDI(limit)`。

```
10 PRINT RNDI(6)+1
```

サイコロ用途の1～6を作る例です。

SIN

Classic 対応

Structured 対応

分類: trigonometric function。構文: `SIN(radians)`。

```
10 PRINT SIN(RAD(90))
```

期待結果は丸め誤差を除き1です。

COS

Classic 対応

Structured 対応

分類: trigonometric function。構文: **COS(radians)**。

```
10 PRINT COS(0)
```

期待結果は1です。

TAN

Classic 対応

Structured 対応

分類: trigonometric function。構文: **TAN(radians)**。

```
10 PRINT TAN(RAD(45))
```

期待結果は丸め誤差を除き1です。

SQR

Classic 対応

Structured 対応

分類: square-root function。構文: **SQR(x)**。

```
10 PRINT SQR(81)
```

期待結果は9です。

ATN

Classic 対応

Structured 対応

分類: 1引数arctangent。構文: **ATN(x)**。

```
10 PRINT DEG(ATN(1))
```

期待結果は約45です。quadrantを区別する場合は**ATAN2(y,x)**を使います。

LOG

Classic 対応

Structured 対応

分類: base-10 logarithm。構文: **LOG(x)**、 $x > 0$ 。

```
10 PRINT LOG(1000)
```

期待結果は3です。v0.92以前のLOGは自然対数でした。旧programの **$Y = \text{LOG}(X)$** が自然対数目的ならv0.93では **$Y = \text{LN}(X)$** へ変更してください。

LN

Classic 対応

Structured 対応

分類: natural logarithm。構文: **LN(x)**、 $x > 0$ 。

```
10 PRINT LN(EXP(1))
```

期待結果は丸め誤差を除き1です。v0.92のLOG相当です。

ASIN

Classic 対応

Structured 対応

分類: arcsine。構文: **ASIN(x)**、 $-1 \sim 1$ 、戻り値radian。

```
10 PRINT DEG(ASIN(1))
```

期待結果は90です。範囲外は**DOMAIN ERROR**です。

ACOS

Classic 対応

Structured 対応

分類: arccosine。構文: **ACOS(x)**、-1～1、戻り値radian。

```
10 PRINT DEG(ACOS(0))
```

期待結果は90です。範囲外は**DOMAIN ERROR**です。

ATAN2

Classic 対応

Structured 対応

分類: 2引数arctangent。構文: **ATAN2(y,x)**。引数順はY、Xです。

```
10 Y=1:X=-1
20 PRINT DEG(ATAN2(Y,X))
```

期待結果は約135です。ATNは1引数でquadrant情報を持ちません。

EXP

Classic 対応

Structured 対応

分類: exponential function。構文: **EXP(x)**。

```
10 PRINT EXP(0)
```

期待結果は1です。

PI

Classic 対応

Structured 対応

分類: numeric constant function。構文: **PI**または**PI()**。

```
10 PRINT 2*PI
```

円周率の2倍を表示します。

RAD

Classic 対応

Structured 対応

分類: angle conversion。構文: **RAD(degrees)**。

```
10 PRINT SIN(RAD(30))
```

期待結果は約0.5です。

DEG

Classic 対応

Structured 対応

分類: angle conversion。構文: **DEG(radians)**。

```
10 PRINT DEG(PI)
```

期待結果は約180です。

SGN

Classic 対応

Structured 対応

分類: sign function。構文: **SGN(x)**。

```
10 PRINT SGN(-8);SGN(0);SGN(8)
```

期待結果は-1、0、1です。

MIN

Classic 対応

Structured 対応

分類: numeric function。構文: **MIN(a,b)**。

```
10 PRINT MIN(23,18)
```

期待結果は18です。

MAX

Classic 対応

Structured 対応

分類: numeric function。構文: MAX(a,b)。

```
10 PRINT MAX(23,18)
```

期待結果は23です。

CLAMP

Classic 対応

Structured 対応

分類: range function。構文: CLAMP(value,low,high)。

```
10 SENSOR=120
20 PRINT CLAMP(SENSOR,0,100)
```

期待結果は100です。meter値の上限・下限処理に使えます。

TIMER

Classic 対応

Structured 対応

分類: elapsed-time function。構文: TIMERまたはTIMER()。

```
10 T=TIMER
20 FOR I=1 TO 1000:NEXT I
30 PRINT TIMER-T
```

処理時間測定の例です。値は実機、clock、処理内容により異なります。

INKEY

Classic 対応

Structured 対応

分類: non-blocking key input。構文: INKEYまたはINKEY()。

```
D0
  K=INKEY
LOOP UNTIL K<>0
PRINT K
```

keyが押されるまでpollし、そのcodeを表示します。終了keyは実際のkey-code表を確認してください。

I2CREAD

Classic 対応

Structured 対応

分類: I2C numeric function。構文: I2CREAD(address,register)。

```
10 SEC=I2CREAD(&H51,&H02)
20 PRINT SEC
```

RTC register読出し例です。device、register map、配線により結果は異なります。

PLAYING

Classic 対応

Structured 対応

分類: audio state function。構文: PLAYING()。

```
10 PLAY "T12005L4 C"
20 IF PLAYING() THEN PRINT "PLAYING"
```

MML playbackがactiveなら真を返します。

POINT

Classic 対応

Structured 対応

分類: graphics query。構文: POINT(x,y)。

```
10 PSET 10,10
20 PRINT POINT(10,10)
```

pixelがnon-blackなら真を返します。

EDIT

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: [EDIT](#)。

```
BASIC> EDIT
```

current programをFull-Screen Editorで開きます。空promptのAlt+Eも同じです。v0.94では、直近のCompile／Runtime Errorとsourceが一致する場合、該当Classic行番号／Structured rowへ移動します。変更済み、Direct実行、sourceなし、保存元削除済みの場合は通常位置で開きます。

LIST

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: [LIST](#)。

```
BASIC> LIST
```

current sourceを表示します。Structuredは行番号なしです。BREAK／Esc／Ctrl-Cで途中停止できます。

RUN

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: [RUN](#)。

```
BASIC> RUN
RUN...
```

current programをcompileして実行します。空promptのAlt+Rも同じです。

NEW

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。

```
BASIC> NEW
OK
```

current programとDirect variablesをclearし、Classicのuntitled programを開始します。Structured新規作成はControl Centerから行います。

CLEAR

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。

```
BASIC> CLEAR
OK (DIRECT VARIABLES CLEARED)
```

Direct variablesだけをclearし、stored sourceは保持します。

CLS command

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。

```
BASIC> CLS
```

console／graphics screenをclearします。

LOAD

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: **LOAD** "name"。

```
BASIC> LOAD "GAMES/DEMO"
```

SDからsourceを読み、全非空行の行番号有無でClassic／Structuredを判定します。SD環境により結果は異なります。

SAVE command

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: **SAVE**または**SAVE** "name"。

```
BASIC> SAVE "TEST"
```

current sourceをSDへ保存します。BASIC statementの**SAVE IMAGE**とは別です。

FILES

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: **FILES** ["directory"]。

```
BASIC> FILES "GAMES"
```

指定directoryのBASIC programを一覧表示します。

DIR

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: **DIR** ["directory"]。

```
BASIC> DIR "MUSIC"
```

指定directoryのfile／directoryを一覧表示します。

SD

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: **SD** [STATUS|REMOUNT]。

```
BASIC> SD STATUS
```

SD状態を表示します。**SD REMOUNT**は明示的な再mountを試みます。

SCREENSHOT

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。構文: **SCREENSHOT** ["name"]。

```
BASIC> SCREENSHOT "RESULT"
```

LCDをBMPとしてSDへ保存します。Alt+Sは自動名で同種の保存を行います。

XRECV

Classic 共通

Structured 共通

分類: REPL transfer command。構文: **XRECV** "name"。

```
BASIC> XRECV "DATA.BAS"
```

XMODEMで1 fileを受信します。接続先とsender設定により結果は異なります。

XSEND

Classic 共通

Structured 共通

分類: REPL transfer command。構文: **XSEND** "name"。

```
BASIC> XSEND "DATA.BAS"
```

XMODEMで1 fileを送信します。

YRECV

Classic 共通

Structured 共通

分類: REPL transfer command。構文: YRECV。

```
BASIC> YRECV
```

YMODEMでsingle／batch fileを受信します。

YSEND

Classic 共通

Structured 共通

分類: REPL transfer command。構文: YSEND "name"。

```
BASIC> YSEND "RESULT.BAS"
```

YMODEMで1 fileを送信します。

DATE

Classic 共通

Structured 共通

分類: REPL clock command。構文: DATE [yyyy-mm-dd]。

```
BASIC> DATE 2026-10-03
```

dateを表示または設定します。RTC構成により結果は異なります。

TIME

Classic 共通

Structured 共通

分類: REPL clock command。構文: TIME [hh:mm:ss]。

```
BASIC> TIME 21:30:00
```

timeを表示または設定します。

DATETIME

Classic 共通

Structured 共通

分類: REPL clock command。構文: DATETIME [YYYYMMDDHHMMSS]。

```
BASIC> DATETIME
```

dateとtimeを表示します。14桁指定で一括設定もできます。

SERIAL

Classic 共通

Structured 共通

分類: REPL console command。構文: SERIAL [ON|OFF|ONLY]。

```
BASIC> SERIAL ON
```

LCD+serial、LCD only、serial onlyを切り替えます。

CONSOLE

Classic 共通

Structured 共通

分類: REPL console command。構文: CONSOLE LCD|BOTH|SERIAL。

```
BASIC> CONSOLE BOTH
```

console output先を明示します。

PROFILE

Classic 共通

Structured 共通

分類: REPL diagnostic command。構文: **PROFILE OFF|COUNT|TIME|SHOW|RESET**。

```
BASIC> PROFILE COUNT
BASIC> RUN
BASIC> PROFILE SHOW
```

VM実行profileを取得します。通常速度は**PROFILE OFF**で測定します。

STANDBY

Classic 共通

Structured 共通

分類: REPL power command。BASIC statementではありません。

```
BASIC> STANDBY
STANDBY - PRESS ANY KEY TO WAKE
```

RAMを保持してLCDを消灯します。USB Storage所有中は拒否されます。

MENU

Classic 共通

Structured 共通

分類: REPL command（BASIC statement不可）。

```
BASIC> MENU
```

Control Centerを開きます。空promptのHOMEまたはAlt+Cも同じです。

DIAGNOSTICS

Classic 共通

Structured 共通

分類: REPL command（BASIC statement不可）。構文: **DIAGNOSTICS**。v0.94で追加。

```
BASIC> DIAGNOSTICS
```

Last Errorを含むSystem Informationの共通snapshotを開きます。左右でpage移動、Rでrefresh、SでUSB CDCへreport送信、Escで戻ります。未発生時はLast Error: Noneです。詳しい操作と診断内容はSystem Manualを参照してください。

HELP

Classic 共通

Structured 共通

分類: REPL command（BASIC statement不可）。

```
BASIC> HELP
```

主要commandとshortcutの概要をconsoleへ表示します。

22.1. v0.94 日時・文字列関数

新しい関数はClassic / Structured / Directで同じparserとVMを使用します。従来の文字列変数の保存長制限は維持されます。

DATE\$

Classic 対応

Structured 対応

分類: BASIC function。Syntax: **DATE\$()** Parameters: なし Return value: YYYY-MM-DD形式の10文字

Description: 既存RTC / software clockの日付を読みます。NTP接続は開始しません。括弧が必須で、括弧なしのDATE\$変数は従来どおり使えます。

Example:

```
10 PRINT DATE$( )
```

Error behavior: 引数があるとARGUMENT COUNT。時刻を取得できない場合はRTC NOT AVAILABLE。

TIMES\$

Classic 対応

Structured 対応

分類: BASIC function。 Syntax: **TIMES\$()** Parameters: なし Return value: HH:MM:SS形式の8文字、24時間制
Description: DATES\$()と同じ時計と設定済みtimezoneを使い、再変換しません。括弧なしのTIMES\$変数は従来どおり使えます。

Example:

```
10 PRINT TIMES()
```

Error behavior: 引数があるとARGUMENT COUNT。時刻を取得できない場合はRTC NOT AVAILABLE。

TRIM\$

Classic 対応

Structured 対応

分類: BASIC function。 Syntax: **TRIM\$(text\$)** Parameters: text\$: 文字列 Return value: 両端の空白を除いた文字列
Description: ASCIIのspace・tab・CR・LF・VT・FFを両端から除きます。途中の空白は保持します。空文字列と空白のみの場合は空文字列になります。

Example:

```
10 PRINT "["+TRIM$(" CPB ")+""]"
```

Error behavior: 引数数はARGUMENT COUNT、型はTYPE MISMATCH、191文字超はSTRING TOO LONG。

LTRIM\$

Classic 対応

Structured 対応

分類: BASIC function。 Syntax: **LTRIM\$(text\$)** Parameters: text\$: 文字列 Return value: 先頭の空白を除いた文字列
Description: TRIM\$と同じASCII空白を先頭から除き、末尾と途中は保持します。

Example:

```
10 PRINT "["+LTRIM$(" CPB ")+""]"
```

Error behavior: TRIM\$と同じ引数数・型・結果長のerrorです。

RTRIM\$

Classic 対応

Structured 対応

分類: BASIC function。 Syntax: **RTRIM\$(text\$)** Parameters: text\$: 文字列 Return value: 末尾の空白を除いた文字列
Description: TRIM\$と同じASCII空白を末尾から除き、先頭と途中は保持します。

Example:

```
10 PRINT "["+RTRIM$(" CPB ")+""]"
```

Error behavior: TRIM\$と同じ引数数・型・結果長のerrorです。

REPLACE\$

Classic 対応

Structured 対応

分類: BASIC function。 Syntax: **REPLACE\$(text\$,find\$,replacement\$)** Parameters: 3引数はすべて文字列 Return value: 置換後の文字列
Description: 大文字小文字を区別し、左から非重複で全置換します。空のfind\$は無変更、空のreplacement\$は削除です。置換結果は再検索しません。

Example:

```
10 PRINT REPLACE$("banana","an","X")
```

Error behavior: 引数数はARGUMENT COUNT、型はTYPE MISMATCH、191文字超はSTRING TOO LONG。

SPACES\$

Classic 対応

Structured 対応

分類: BASIC function。 Syntax: **SPACES\$(count)** Parameters: count: 0以上192未満の有限数値。小数部は切り捨て
Return value: count個のASCII space Description: 0なら空文字列、最大191文字です。SPC / STRING\$は変更しません。

Example:

```
10 PRINT "A"+SPACES$(3)+"B"
```

Error behavior: 引数数はARGUMENT COUNT、型はTYPE MISMATCH、負数・非有限値・192以上はBAD COUNT。

22.2. v0.94 共通REPL commands

INFO / LASTERRORはClassic / Structured共通です。Programを変更せず、REPLから現在の状態や記録済みerrorを確認します。以下のcommandを保存済みBASIC source内のstatementとして使うことはできません。

INFO

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。 Syntax: **INFO** Parameters: なし Return value: なし
Description: System Information共通のcollectorからversion / mode / program / memory / SD / Wi-Fi / audioを表示。heap取得不可はN/A。状態は変更しません。

Example:

```
BASIC> INFO
```

Error behavior: snapshot確保不可はOUT OF MEMORY。program / Last Errorは保持します。

LASTERROR

Classic 共通

Structured 共通

分類: REPL command (BASIC statement不可)。 Syntax: **LASTERROR** Parameters: なし Return value: なし
Description: 既存Last Error記録のtype、message、program、source mode、Classic line / Structured row、FUNCTIONと取得済みtraceを表示します。Compile / Runtime / Directが対象です。成功やBREAKでは過去のerror記録を消しません。DIAGNOSTICS / Serialと同じreportを使用し、ソース本文は転送しません。

Example:

```
BASIC> RUN
BASIC> LASTERROR
```

Error behavior: 記録がなければNo error recorded.。snapshot用memoryを確保できなければOUT OF MEMORY。programと記録を変更しません。

23. Quick Reference (alphabetical)

Statements

Assignment / LET, BEEP, Block IF / ELSEIF / ELSE / END IF, BOX, CASE, CASE ELSE, CIRCLE, CLS statement, COLOR, COLORHSV, DATA, DIM, DO / LOOP, END, END FUNCTION, END SELECT, EXIT DO, EXIT FOR, FLUSH, FOR / NEXT, FUNCTION, GDEF, GLOBAL, GLOCATE, GOSUB, GOTO, GPALETTE, GPRINT, I2C SCAN, I2CWRITE, INPUT, LINE, LOADIMAGE, LOCATE, ON ... GOSUB, ON ... GOTO, PAINT, PAUSE, PLAY, PRINT, PSET, RANDOMIZE, READ, REM / apostrophe comment, RESTORE, RETURN - Classic GOSUB return, RETURN expression - Structured FUNCTION return, SAVEIMAGE / SAVE IMAGE, SCREEN, SELECT CASE, Single-line IF, SLEEP, STOP, WAVPAUSE, WAVPLAY, WAVRESUME, WAVSTOP, WHILE / WEND

Functions

ABS, ACOS, ASC, ASIN, ATAN2, ATN, CHR\$, CLAMP, COS, DEG, EXP, I2CREAD, INKEY, INSTR, INT, LEFT\$, LEN, LN, LOG, MAX, MID\$, MIN, PI, PLAYING, POINT, RAD, RND, RNDI, RIGHT\$, SGN, SIN, SPC, SQR, STR\$, STRING\$, TRIM\$, LTRIM\$, RTRIM\$, REPLACE\$, SPACE\$, DATE\$, TIME\$, TAB, TAN, TIMER, VAL

Operators

算術: +, -, *, /, ^, MOD, 整数除算\。shift/bit: <<, >>, XOR。比較: =, <>, <, <=, >, >=。logical: NOT, AND, OR。v0.93追加operatorの例は「Operators \, <<, >>, XOR」を参照してください。

REPL commands

CLEAR, CLS command, CONSOLE, DATE, DATETIME, DIAGNOSTICS, DIR, EDIT, FILES, HELP, INFO, LASTERROR, LIST, LOAD, MENU, NEW, PROFILE, RUN, SAVE command, SCREENSHOT, SD, SERIAL, STANDBY, TIME, XRECV, XSEND, YRECV, YSEND